

山东大学__计算机科学与技术__学院

大数据分析实践 课程实验报告

学 号 : 202300130059	姓名: 林子洋	班级: 23 数据
实验题目: Canis/Cast/Libra 实践		
实验学时: 2	实验日期: 2025.11.1	
实验目的: 1: 掌握 Canis 框架的可视化开发流程, 实现散点图绘制与交互功能。 2: 验证 Canis 与 D3.js 结合的可行性, 实现数据点高亮、tooltip 展示等交互效果。 3: 理解可视化组件化设计思路, 提升数据可视化应用开发能力。		
硬件环境: 计算机一台		
软件环境: Linux, Canis, D3.js v7 依赖库		

实验步骤与内容：

- 1：环境准备：引入 Canis 核心库与 D3.js 依赖，配置基础 HTML 结构与样式。
- 2：数据处理：生成含 4 个类别、120 个数据点的示例数据，包含坐标、类别等属性。
- 3：组件实现：创建可视化图层，设计坐标轴、图例与散点图绘制逻辑。
- 4：交互开发：实现悬停显示详情、点击高亮同类别、拖拽平移、滚轮缩放功能。
- 5：测试运行：验证可视化效果与交互功能的完整性和稳定性。

实验代码参考如下：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Canis 散点图可视化</title>
  <!-- 引入 Canis 核心库 -->
  <script
src="https://cdn.jsdelivr.net/npm/canisjs/dist/canis.min.js"></scri
```

pt>

<!-- 引入 D3 依赖 -->

<script src="https://d3js.org/d3.v7.min.js"></script>

<style>

* {

box-sizing: border-box;

margin: 0;

padding: 0;

}

body {

font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;

background: #f5f5f5;

padding: 20px;

min-height: 100vh;

}

.container {

max-width: 1000px;

margin: 0 auto;

background: white;

border-radius: 8px;

box-shadow: 0 2px 10px rgba(0,0,0,0.1);

```
padding: 30px;
}
h1 {
  color: #2d3748;
  text-align: center;
  margin-bottom: 8px;
  font-size: 28px;
  font-weight: 600;
}
.subtitle {
  text-align: center;
  color: #718096;
  margin-bottom: 30px;
  font-size: 14px;
}
#visualization {
  width: 100%;
  height: 500px;
  margin: 0 auto;
  position: relative;
}
.info-panel {
```

```
margin-top: 25px;
padding: 15px;
background: #f8f9fa;
border-radius: 6px;
border-left: 4px solid #4299e1;
}

.info-panel h3 {
  color: #2d3748;
  margin-bottom: 10px;
  font-size: 16px;
}

.info-panel ul {
  list-style: none;
  padding-left: 10px;
}

.info-panel li {
  margin: 6px 0;
  color: #4a5568;
  font-size: 14px;
  display: flex;
  align-items: center;
}
```

```
.info-panel li::before {
    content: "•";
    color: #4299e1;
    font-weight: bold;
    margin-right: 8px;
}

.info-panel li strong {
    color: #2d3748;
}

.tooltip {
    position: absolute;
    background: rgba(0,0,0,0.85);
    color: white;
    padding: 12px 16px;
    border-radius: 6px;
    pointer-events: none;
    font-size: 13px;
    z-index: 9999;
    min-width: 200px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.2);
}

.legend {
```

```
    position: absolute;
    top: 20px;
    right: 20px;
    background: rgba(255,255,255,0.9);
    padding: 10px 15px;
    border-radius: 6px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

.legend-item {
    display: flex;
    align-items: center;
    margin: 5px 0;
    font-size: 12px;
    color: #4a5568;
}

.legend-color {
    width: 12px;
    height: 12px;
    border-radius: 50%;
    margin-right: 8px;
}

</style>
```

```
</head>
<body>
  <div class="container">
    <h1>Canis 交互式散点图</h1>
    <p class="subtitle">基于 Canis 框架的可视化实践</p>

    <div id="visualization"></div>

    <div class="info-panel">
      <h3>交互功能</h3>
      <ul>
        <li><strong>悬停</strong>： 查看数据点详细信息</li>
        <li><strong>点击</strong>： 高亮同类别数据点</li>
        <li><strong>拖拽</strong>： 平移视图（按住鼠标左
        键） </li>
        <li><strong>滚轮</strong>： 缩放视图</li>
      </ul>
    </div>
  </div>

  <script>
```

```
// Canis 应用初始化
```

```
const app = new Canis.Application({  
  container: document.getElementById('visualization'),  
  width: 800,  
  height: 500,  
  backgroundColor: '#ffffff'  
});
```

```
// 配置参数
```

```
const config = {  
  margin: { top: 40, right: 80, bottom: 60, left: 60 },  
  categories: ['A', 'B', 'C', 'D'],  
  colors: ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A'],  
  pointRadius: 6  
};
```

```
// 颜色比例尺
```

```
const colorScale = d3.scaleOrdinal()  
  .domain(config.categories)  
  .range(config.colors);
```

```
// 生成示例数据
```

```
function generateData() {  
  const data = [];  
  config.categories.forEach((cat, idx) => {  
    for (let i = 0; i < 30; i++) {  
      const baseX = 100 + idx * 200;  
      const baseY = 100 + idx * 100;  
      data.push({  
        id: `${cat}-${i}`,  
        x: baseX + (Math.random() - 0.5) * 150,  
        y: baseY + (Math.random() - 0.5) * 150,  
        category: cat,  
        value: Math.floor(Math.random() * 100),  
        label: `${cat}-${i+1}`  
      });  
    }  
  });  
  return data;  
}
```

// 创建数据层

```
const dataLayer = new Canis.Layer('scatter-layer', {  
  x: config.margin.left,
```

```
y: config.margin.top,
width: app.width - config.margin.left - config.margin.right,
      height: app.height - config.margin.top -
config.margin.bottom,
      interactive: true
});
app.addLayer(dataLayer);

// 创建坐标轴
function createAxes(data) {
  // X 轴
  const xScale = d3.scaleLinear()
    .domain(d3.extent(data, d => d.x))
    .nice()
    .range([0, dataLayer.width]);

  const xAxis = d3.axisBottom(xScale)
    .ticks(8)
    .tickSize(-dataLayer.height)
    .tickPadding(10);

  const xAxisGroup = dataLayer.svg.append('g')
```

```
.attr('class', 'axis x-axis')
.attr('transform', `translate(0, ${dataLayer.height})`)
.call(xAxis);

xAxisGroup.append('text')
.attr('class', 'axis-label')
.attr('x', dataLayer.width / 2)
.attr('y', 45)
.attr('fill', '#4a5568')
.style('text-anchor', 'middle')
.text('X 坐标');

// Y 轴
const yScale = d3.scaleLinear()
  .domain(d3.extent(data, d => d.y))
  .nice()
  .range([dataLayer.height, 0]);

const yAxis = d3.axisLeft(yScale)
  .ticks(8)
  .tickSize(-dataLayer.width)
  .tickPadding(10);
```

```
const yAxisGroup = dataLayer.svg.append('g')  
  .attr('class', 'axis y-axis')  
  .call(yAxis);
```

```
yAxisGroup.append('text')  
  .attr('class', 'axis-label')  
  .attr('transform', 'rotate(-90)')  
  .attr('x', -dataLayer.height / 2)  
  .attr('y', -45)  
  .attr('fill', '#4a5568')  
  .style('text-anchor', 'middle')  
  .text('Y 坐标');
```

```
// 网格线样式
```

```
dataLayer.svg.selectAll('.axis line')  
  .attr('stroke', '#e2e8f0')  
  .attr('stroke-width', 1);
```

```
dataLayer.svg.selectAll('.axis path')  
  .attr('stroke', '#cbd5e0')  
  .attr('stroke-width', 1);
```

```
    return { xScale, yScale };
  }

// 创建图例
function createLegend() {
  const legend = document.createElement('div');
  legend.className = 'legend';

  config.categories.forEach(cat => {
    const item = document.createElement('div');
    item.className = 'legend-item';

    const colorBox = document.createElement('div');
    colorBox.className = 'legend-color';
    colorBox.style.backgroundColor = colorScale(cat);

    const label = document.createElement('span');
    label.textContent = `类别 ${cat}`;

    item.appendChild(colorBox);
    item.appendChild(label);
  });
}
```

```
        legend.appendChild(item);
    });

    document.getElementById('visualization').appendChild(legend);
}

// 绘制散点图
function drawScatterPlot(data, scales) {
    const { xScale, yScale } = scales;

    // 创建数据点组
    const points = dataLayer.svg.append('g')
        .attr('class', 'data-points');

    // 绘制数据点
    const pointElements = points.selectAll('circle')
        .data(data)
        .enter()
        .append('circle')
        .attr('class', d => `point point-${d.category}`)
        .attr('cx', d => xScale(d.x))
```

```
.attr('cy', d => yScale(d.y))  
.attr('r', config.pointRadius)  
.attr('fill', d => colorScale(d.category))  
.attr('fill-opacity', 0.7)  
.attr('stroke', d => colorScale(d.category))  
.attr('stroke-width', 1.5)  
.style('cursor', 'pointer');
```

// 交互效果：悬停显示 tooltip

```
const tooltip = d3.select('body').append('div')  
.attr('class', 'tooltip')  
.style('opacity', 0);
```

pointElements

```
.on('mouseover', function(event, d) {  
  tooltip.transition()  
    .duration(200)  
    .style('opacity', 1);
```

```
  tooltip.html(`  
    <div style="margin-bottom: 4px; font-weight: 600;  
color: #4299e1;">${d.label}</div>
```

```

        <div><span style="color: #94a3b8;">类别:</span>
    ${d.category}</div>

        <div><span style="color: #94a3b8;">X 值:</span>
    ${d.x.toFixed(2)}</div>

        <div><span style="color: #94a3b8;">Y 值:</span>
    ${d.y.toFixed(2)}</div>

        <div><span style="color: #94a3b8;">数值:</span>
    ${d.value}</div>
    `)
    .style('left', (event.pageX + 10) + 'px')
    .style('top', (event.pageY - 10) + 'px');

// 悬停高亮
d3.selectAll(`.point-${d.category}`)
    .attr('fill-opacity', 1)
    .attr('r', config.pointRadius + 1);
})
.on('mouseout', function(event, d) {
    tooltip.transition()
        .duration(200)
        .style('opacity', 0);

```

```
// 恢复默认样式
d3.selectAll(`.point-${d.category}`)
  .attr('fill-opacity', 0.7)
  .attr('r', config.pointRadius);
})
.on('click', function(event, d) {
  // 点击高亮同类别
  const isActive = d3.select(this).attr('data-active') ===
'true';

  // 重置所有点
  d3.selectAll('.point')
    .attr('fill-opacity', 0.7)
    .attr('stroke-width', 1.5)
    .attr('data-active', 'false');

  if (!isActive) {
    // 高亮当前类别
    d3.selectAll(`.point-${d.category}`)
      .attr('fill-opacity', 1)
      .attr('stroke-width', 3)
      .attr('data-active', 'true');
```

```
    }  
  });  
  
  // 添加 Canis 的平移缩放交互  
  const zoom = d3.zoom()  
    .scaleExtent([0.5, 3])  
    .on('zoom', function(event) {  
      points.attr('transform', event.transform);  
      dataLayer.svg.select('.x-axis').call(xAxis.scale(event.transform.rescaleX(xScale)));  
      dataLayer.svg.select('.y-axis').call(yAxis.scale(event.transform.rescaleY(yScale)));  
    });  
  
  dataLayer.svg.call(zoom);  
}  
  
// 初始化应用  
async function initApp() {  
  // 生成数据  
  const data = generateData();
```

```
// 创建坐标轴
const scales = createAxes(data);

// 创建图例
createLegend();

// 绘制散点图
drawScatterPlot(data, scales);

// 启动 Canis 应用
app.start();
}

// 页面加载完成后初始化
document.addEventListener('DOMContentLoaded', initApp);
</script>
</body>
</html>
```

结论分析与体会：

分析：

- 1：功能验证：成功实现散点图可视化与多维度交互，数据展示清晰、交互响应流畅。
- 2：技术适配：Canis 与 D3.js 兼容良好，组件化设计提升了代码可维护性。
- 3：优势体现：Canis 的图层管理与交互支持简化了可视化开发流程，适配多设备场景。

体会：

1. Canis 框架降低了可视化开发门槛，组件化与交互 API 设计实用高效。
2. 数据可视化需平衡视觉呈现与交互体验，合理的布局与样式提升用户体验。
3. 多库结合（Canis+D3）可互补优势，适合快速开发功能完善的可视化应用。